

Object-Oriented Programming in Java

Emily Lucia Antosch

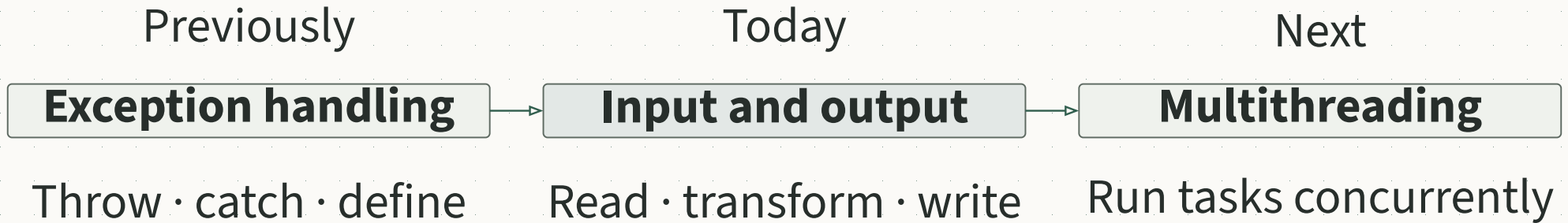
18.09.2026

Today's route

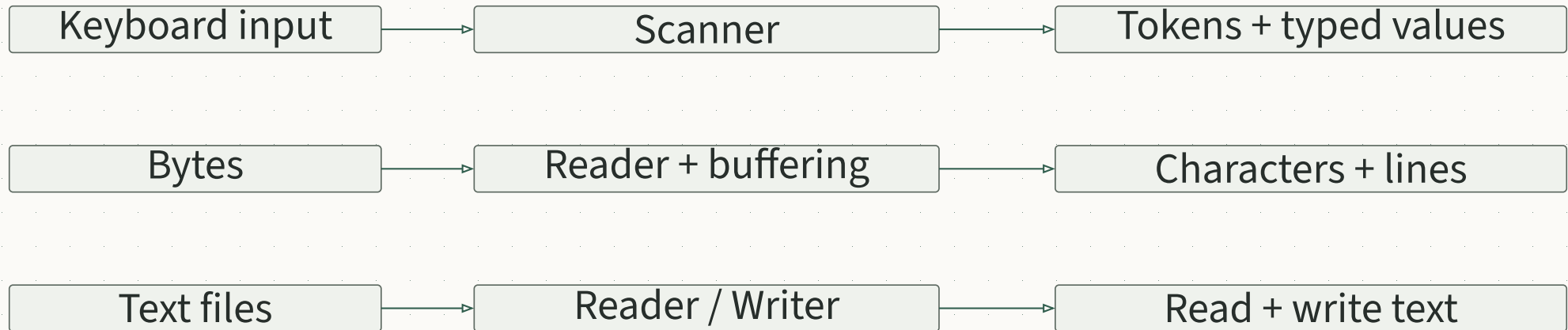
1. Introduction	2
2. Streams and Screen Output	5
3. Keyboard Input	11
4. Byte and Character Streams	25
5. Files	32
6. License Notice	41

1. Introduction

From Exceptions to Input and Output



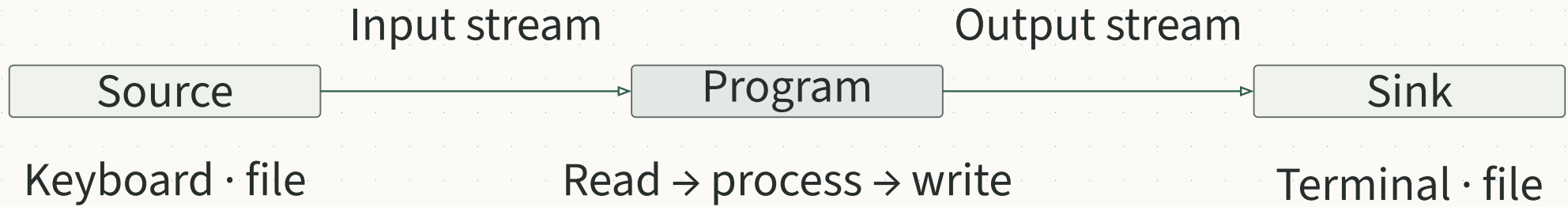
Read, Transform, Write



Examples omit imports from `java.io`, `java.util` and `java.nio.charset`.

2. Streams and Screen Output

Input and Output Are Relative to the Program



Read `System.out.println()`



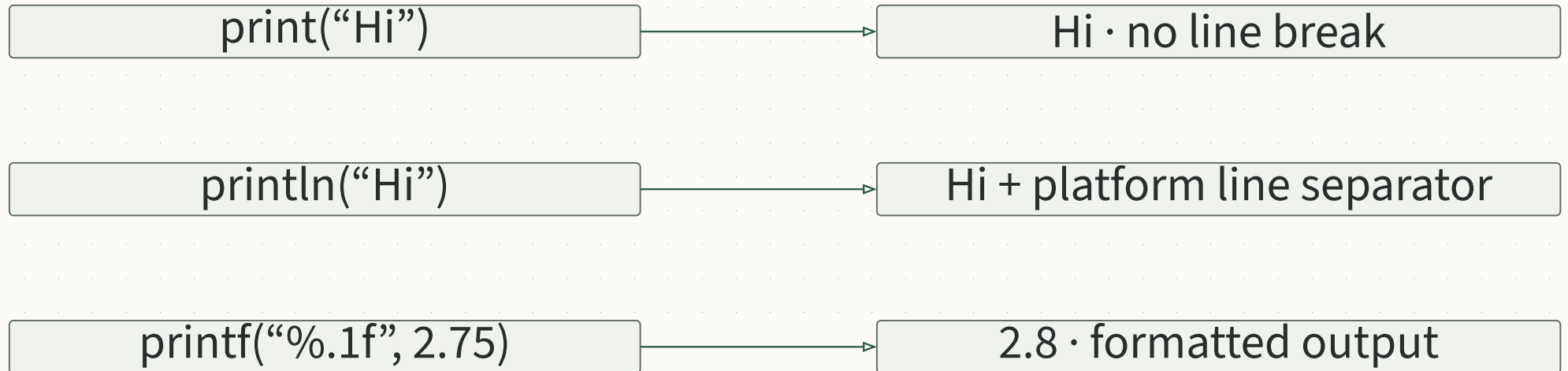
Class

Static field → `PrintStream`

Instance method call

`System.out` normally writes to the terminal; it can be redirected

Choose an Output Method



`PrintStream.format` works like `printf` · number formatting depends on locale

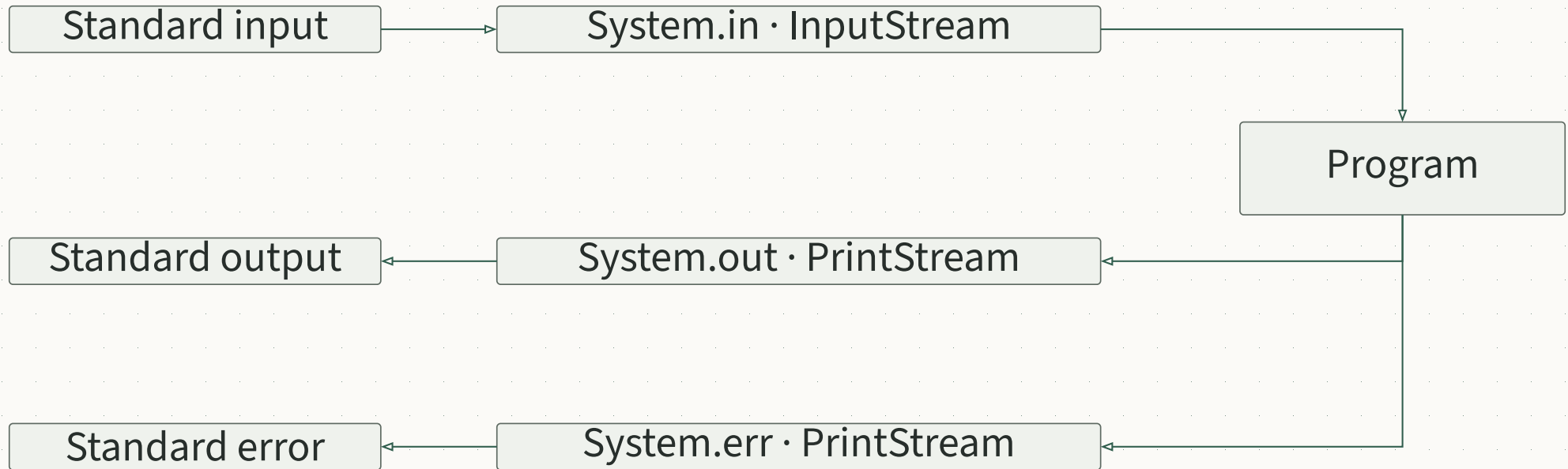
Predict the Output

```
1 public static void main(String[] args) {  
2     double hawaii = 15.97;  
3     double hamburg = 22.71;  
4     String.format(Locale.ROOT, "Hawaii: %.1f °C", hawaii);  
5     System.out.printf(Locale.ROOT, "Hamburg: %.1f °C", hamburg);  
6 }
```

Java

Which call prints? Which returns a String? Is there a line break?

Three Standard Streams

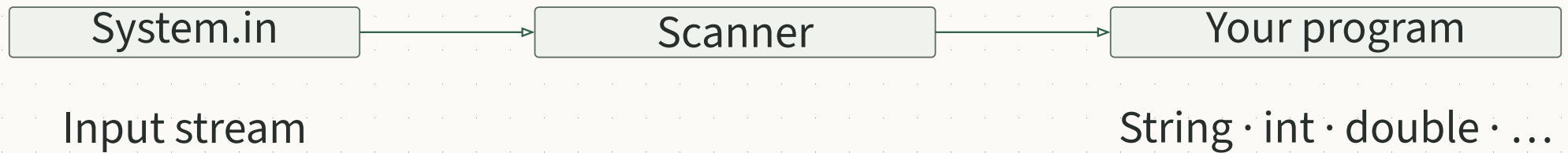


Usually keyboard / terminal · each stream can be redirected independently

3. Keyboard Input

Scanner Splits and Parses Input

Split tokens + parse values



`close()` closes the underlying stream, including `System.in`

These one-shot examples close `System.in` at the end. Keep a shared `Scanner` open while the application still needs input.

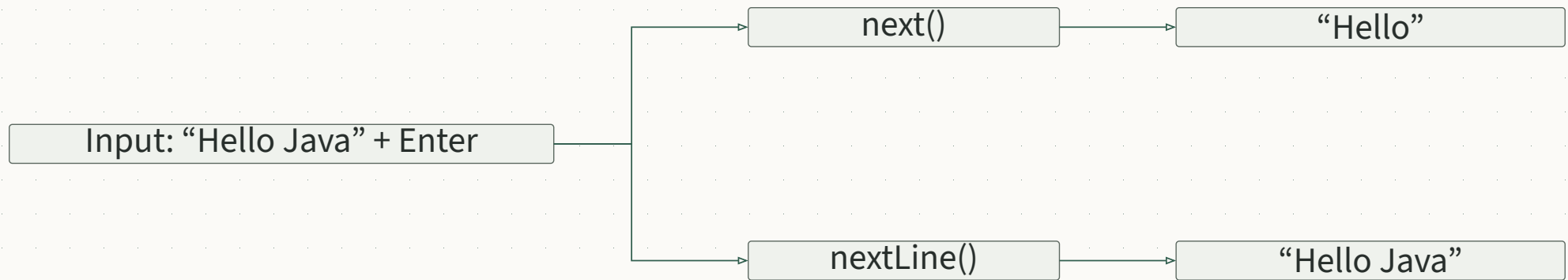
Read a Line

```
1 public static void main(String[] args) {  
2     Scanner scanner = new Scanner(System.in);  
3     System.out.print("Please enter a sentence: ");  
4     System.out.println(scanner.nextLine());  
5     scanner.close();  
6 }
```

Java

Replace `nextLine()` with `next()`. What changes?

Tokens Are Not Lines

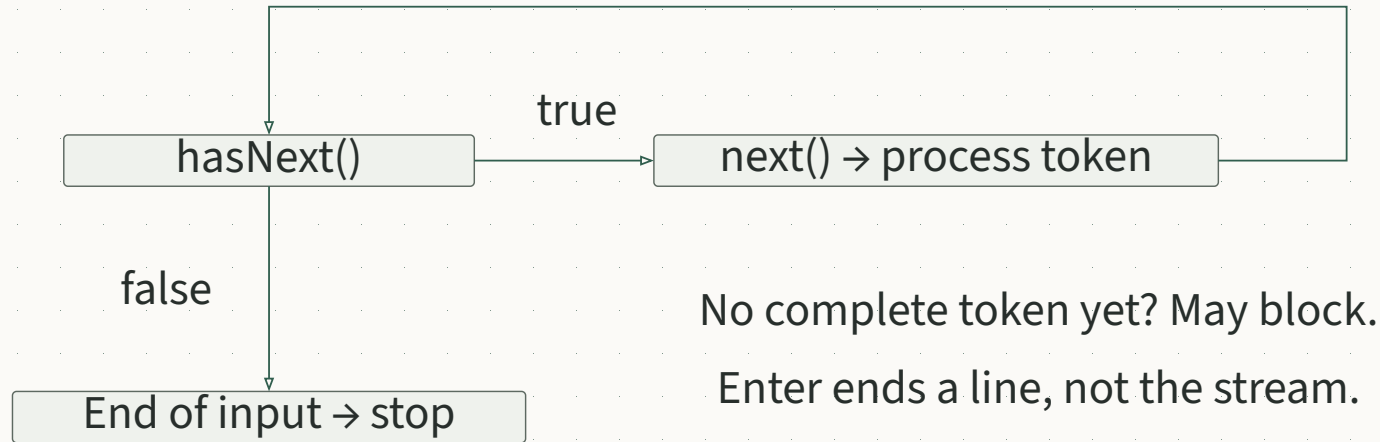


Fresh Scanner for each path • `nextLine` consumes the line separator

Default token delimiters: whitespace • change with `useDelimiter(...)`

After `nextInt()`, `nextLine()` reads the rest of the current line—possibly empty.

Read Until End of Input



`next()` at end of input throws `NoSuchElementException`



Task 1

Read and print each token with `next()`. Stop at end of input.

A Token Loop

```
1 public static void main(String[] args) {  
2     Scanner scanner = new Scanner(System.in);  
3     while (scanner.hasNext()) {  
4         System.out.println(scanner.next());  
5     }  
6     scanner.close();  
7 }
```

Java

What changes if the loop condition is true? Send EOF: Ctrl-D on an empty line (Unix), or Ctrl-Z then Enter (Windows).

Parse a Typed Value

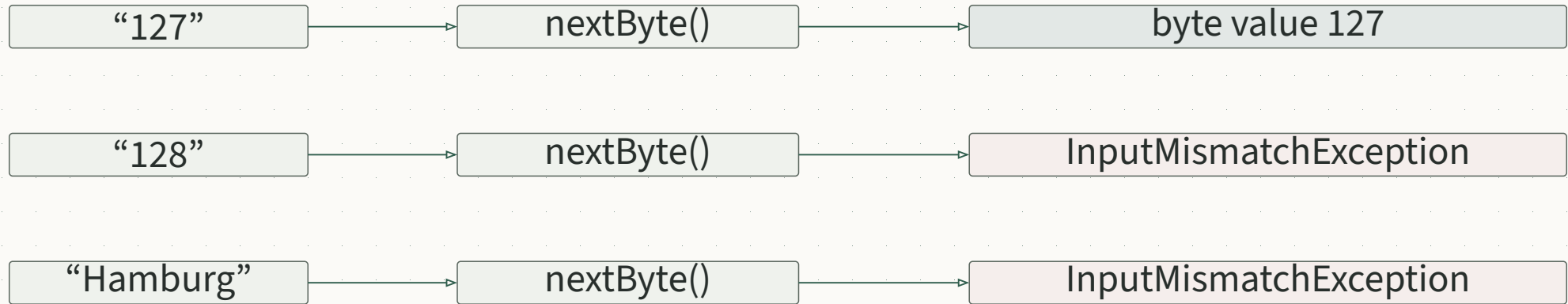
Predict the result for 127, 128 and Hamburg.

```
1 public static void main(String[] args) {  
2     Scanner scanner = new Scanner(System.in);  
3     System.out.print("Please enter a byte value: ");  
4     System.out.println("Entered: " + scanner.nextByte());  
5     scanner.close();  
6 }
```

Java

Other pairs: `nextInt() / hasNextInt()`, `nextDouble() / hasNextDouble()`, `nextBoolean() / hasNextBoolean()`.

A Token Must Fit the Type



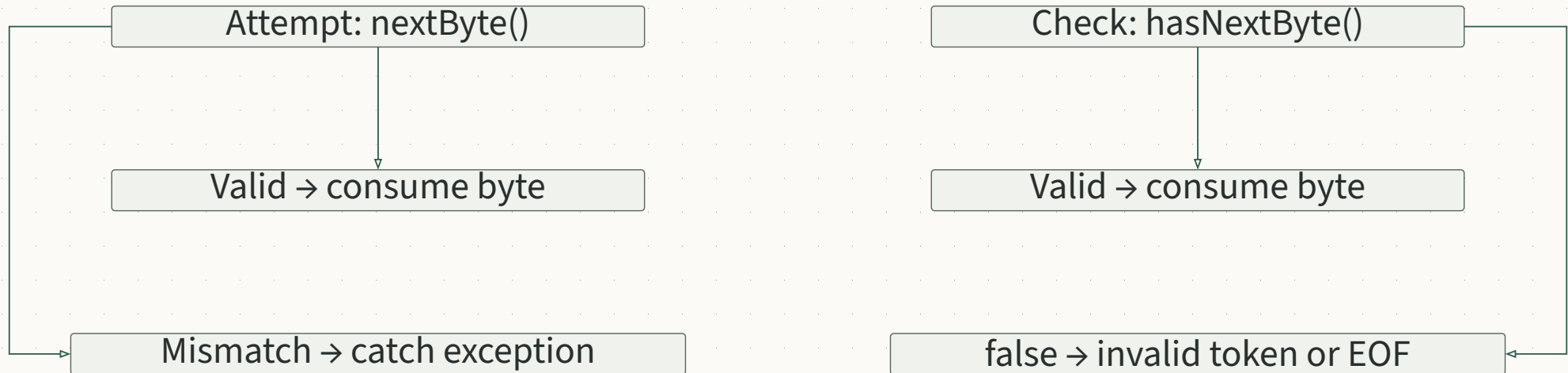
Valid byte range: $-128 \dots 127$ · mismatch leaves the token unread



Task 2

Handle invalid input in two ways: catch the exception, or check before reading.

Two Ways to Handle a Mismatch



`InputMismatchException` is unchecked · handle it or validate first

`hasNextByte()` does not consume input. Before retrying, consume an invalid token with `next()`.

Catch the Exception

```
1  public static void main(String[] args) {
2      Scanner scanner = new Scanner(System.in);
3      try {
4          System.out.println("Entered: " + scanner.nextByte());
5      } catch (InputMismatchException e) {
6          System.out.println("Not a byte value: " + scanner.next());
7      } finally {
8          scanner.close();
9      }
10 }
```

Java

This version assumes a token is available. EOF throws `NoSuchElementException`.

Check Before Reading

```
1  public static void main(String[] args) {
2      Scanner scanner = new Scanner(System.in);
3      if (scanner.hasNextByte()) {
4          System.out.println("Entered: " + scanner.nextByte());
5      } else if (scanner.hasNext()) {
6          System.out.println("Not a byte value: " + scanner.next());
7      } else {
8          System.out.println("End of input.");
9      }
10     scanner.close();
11 }
```

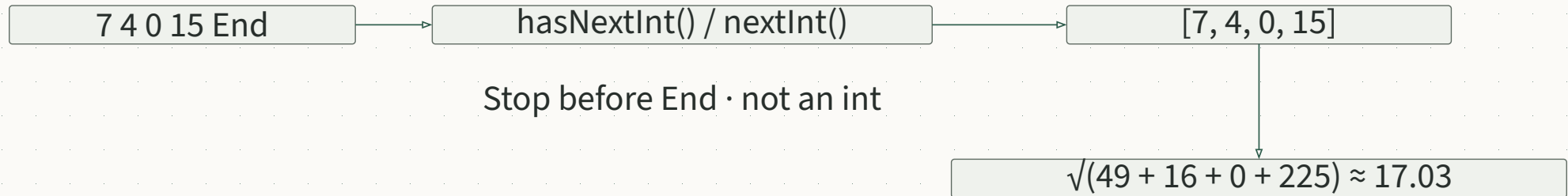
Java

Task: Read a Vector



Task 3

Read int components until a non-integer token or EOF. Print the vector and its magnitude.



Collect the Components

```
1 public static void main(String[] args) {  
2     Scanner scanner = new Scanner(System.in);  
3     ArrayList<Integer> vector = new ArrayList<>();  
4     while (scanner.hasNextInt()) {  
5         vector.add(scanner.nextInt());  
6     }  
7     scanner.close();  
8     printVector(vector);  
9 }
```

Java

Add the method from the next slide to the same class.

Compute the Magnitude

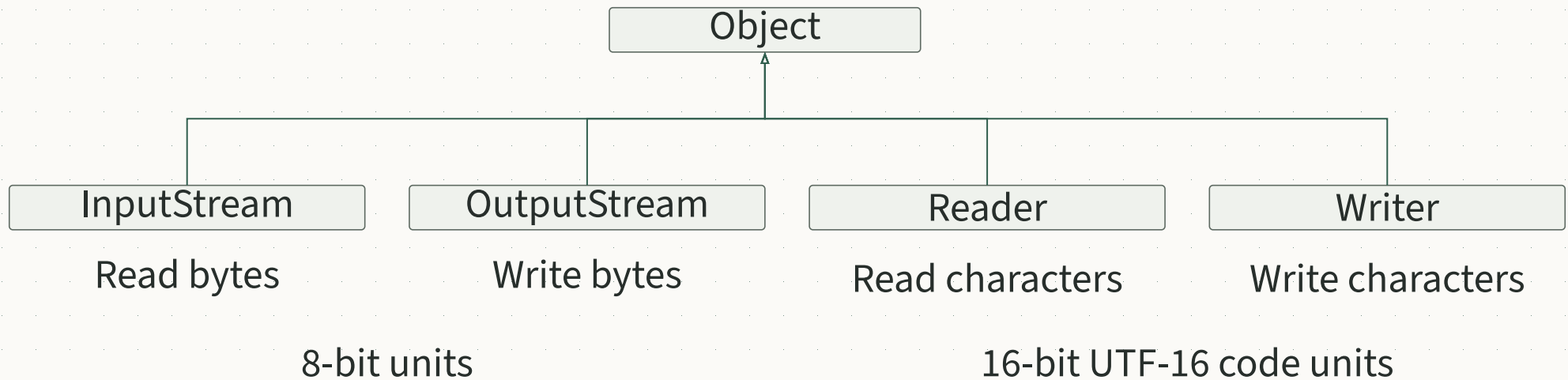
```
1 static void printVector(ArrayList<Integer> vector) {  
2     double sumOfSquares = 0;  
3     for (int component : vector) {  
4         sumOfSquares += (double) component * component;  
5     }  
6     System.out.println("a = " + vector + "^T");  
7     System.out.printf(Locale.ROOT, "||a|| = %.2f%n",  
8         Math.sqrt(sumOfSquares));  
9 }
```

Java

Convert before multiplication to avoid int overflow. An empty vector has magnitude 0.

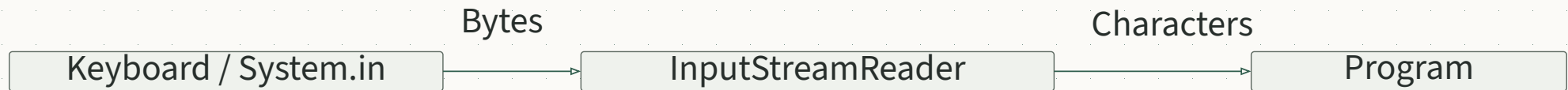
4. Byte and Character Streams

Bytes and Characters Have Different APIs



All four are abstract classes • a Unicode character may need two char values

Decode Bytes into Characters



Decode using the input charset

Example: UTF-8 C3 A4 → ä → UTF-16 00E4

Use the charset that matches the input. Scanner is still convenient for keyboard input; this chain shows the underlying layers.

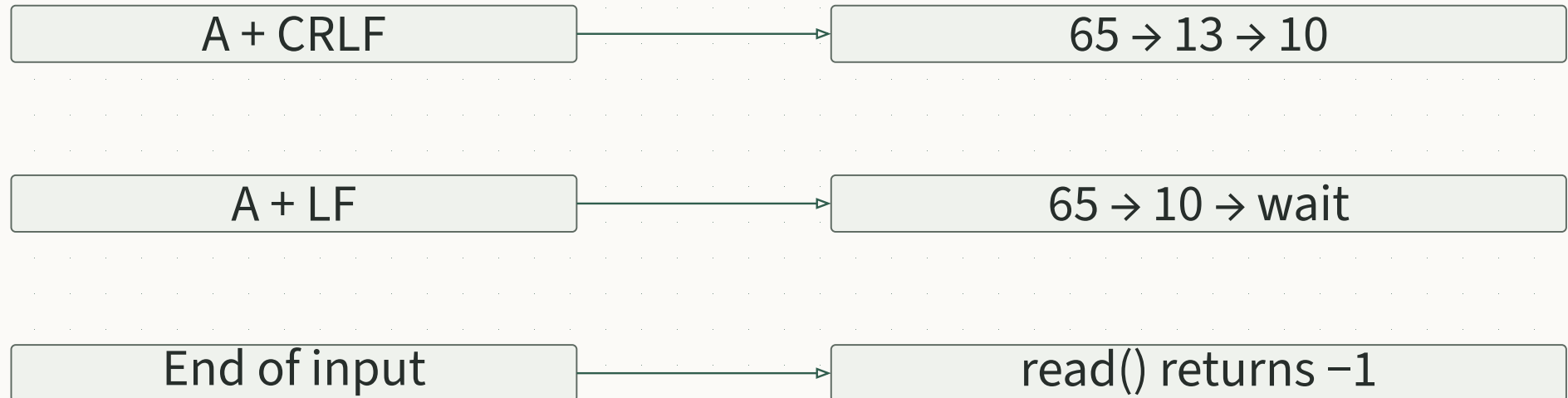
Read Individual Character Values

```
1 public static void main(String[] args) throws IOException {
2     InputStreamReader reader = new InputStreamReader(System.in);
3     System.out.print("Please enter a character: ");
4     System.out.println(reader.read());
5     System.out.println(reader.read());
6     System.out.println(reader.read());
7     reader.close();
8 }
```

Java

`read()` returns a UTF-16 code unit as an `int`, or `-1` at EOF. What happens after typing A and pressing Enter?

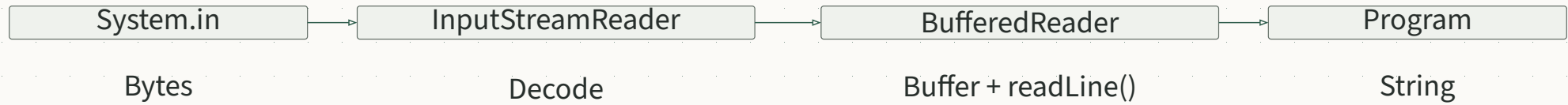
Enter Does Not Always Mean 13, 10



`read()` returns an `int` · line endings depend on the terminal / platform

With LF, a third read may block until more input or EOF

Buffer Characters, Read Lines



readLine() removes the line separator · null means end of input



Task 4

Read two lines with `BufferedReader`, then print both. Close the outer reader.

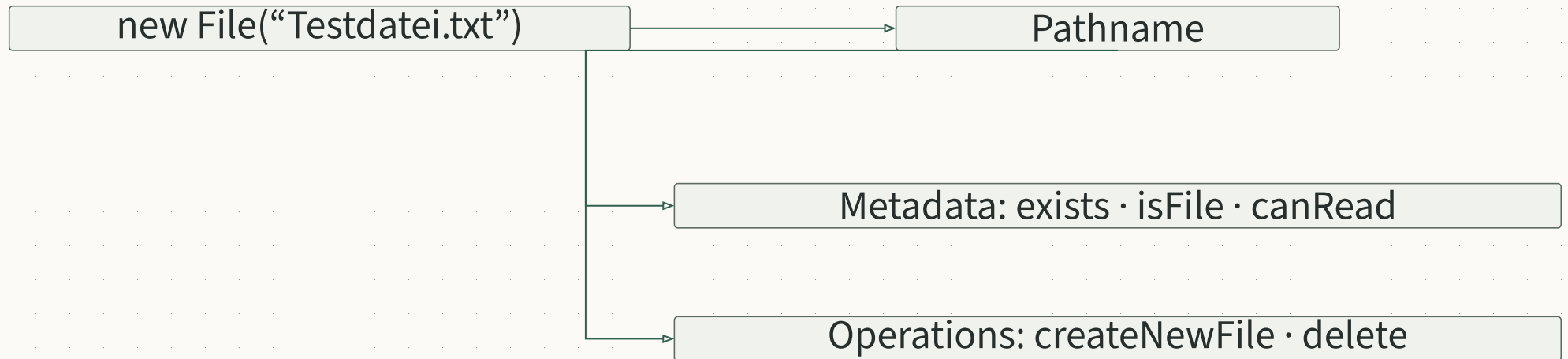
Read Two Lines

```
1  public static void main(String[] args) throws IOException {
2      InputStreamReader reader = new InputStreamReader(System.in);
3      BufferedReader bufferedReader = new BufferedReader(reader);
4      System.out.print("First line: ");
5      String line1 = bufferedReader.readLine();
6      System.out.print("Second line: ");
7      String line2 = bufferedReader.readLine();
8      System.out.println(line1);
9      System.out.println(line2);
10     bufferedReader.close();
11 }
```

Java

5. Files

A File Object Represents a Path



File does not contain file data · streams read / write the contents

Relative paths resolve from the process working directory

Create, Inspect, Delete

```
1  public static void main(String[] args) throws IOException {
2      File file = new File("Testdatei.txt");
3      if (file.createNewFile()) {
4          try {
5              System.out.println("Read: " + file.canRead());
6              System.out.println("Write: " + file.canWrite());
7          } finally {
8              if (!file.delete()) {
9                  throw new IOException("Could not delete " + file);
10             }
11         }
12     } else {
13         System.out.println("Already exists; left unchanged.");
14     }
15 }
```

Java

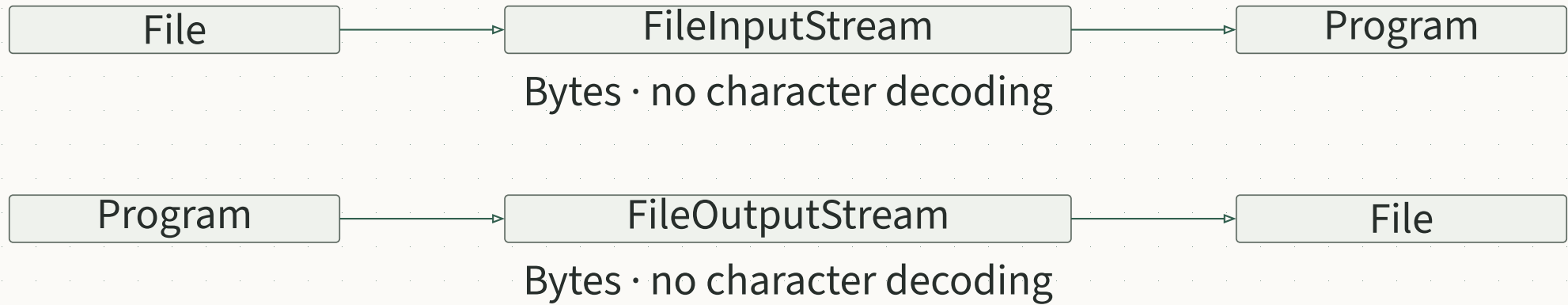
List a Directory

```
1 public static void main(String[] args) throws IOException {  
2     File directory = new File(".");  
3     String[] children = directory.list();  
4     if (children == null) {  
5         throw new IOException("Cannot list " + directory);  
6     }  
7     for (String child : children) {  
8         System.out.println(child);  
9     }  
10 }
```

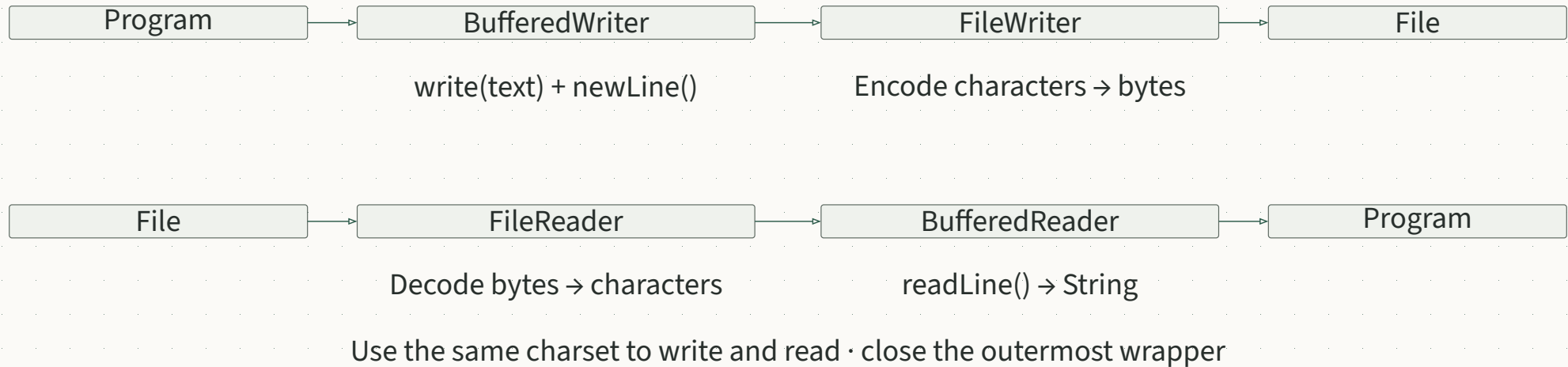
Java

`list()` returns null if the path is not a directory or an I/O error occurs.

Read and Write Raw Bytes



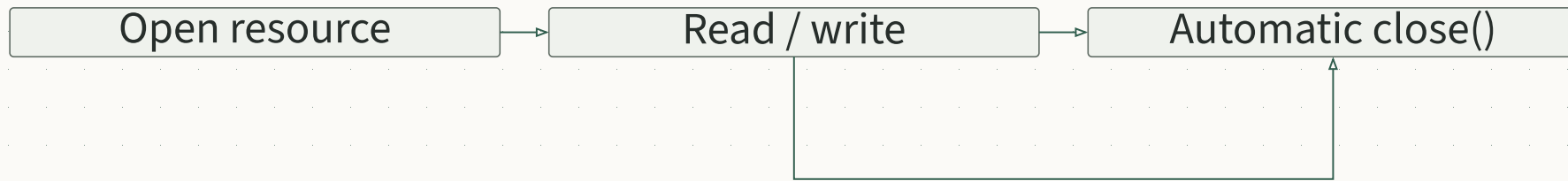
Chain Text File Streams



`BufferedWriter.newLine()` writes only the platform line separator, not a whole line.

Close Resources on Every Exit

try-with-resources



Also on exception or return

```
1 try (BufferedReader reader = new BufferedReader(  
2     new FileReader("Testdatei.txt", StandardCharsets.UTF_8))) {  
3     System.out.println(reader.readLine());  
4 }
```

Java

Closing the wrapper closes its underlying reader. FileReader / FileWriter charset constructors require Java 11+.

Write a Text File



Task 5

Write two lines to a file, then read and print them in a second program.

```
1 public static void main(String[] args) throws IOException {
2     try (BufferedWriter writer = new BufferedWriter(
3         new FileWriter("Testdatei.txt", StandardCharsets.UTF_8))) {
4         writer.write("This is the first line.");
5         writer.newLine();
6         writer.write("And here comes the second line.");
7         writer.newLine();
8     }
9 }
```

Java

This overwrites an existing file. Closing flushes buffered output.

Read Until EOF, Not Until ready() Is False

```
1 public static void main(String[] args) throws IOException {
2     try (BufferedReader reader = new BufferedReader(
3         new FileReader("Testdatei.txt", StandardCharsets.UTF_8))) {
4         String line;
5         while ((line = reader.readLine()) != null) {
6             System.out.println(line);
7         }
8     }
9 }
```

Java

ready() checks whether reading can proceed without blocking. It is not an EOF test.

6. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0.
- Based on teaching material by Prof. Dr. Marc Hensel, supplied privately.
- Text, diagrams and layout adapted; the private source cannot be linked.